Compte rendu - Choix de la méthode de gestion de projet

Contexte du projet :

Projetglobe consiste à refaire une grande partie de l'intranet de notre organisme de formation

L'intranet doit être utilisé à la fois par les etudiant mais aussi par l'equipe pedagogique

Les besoins sont assez nombreux et vont surement évoluer dans le temps car le projet est prevu sur plusieurs mois

Le directeur souhaite aussi pouvoir faire évoluer le projet en plusieurs versions et il envisage meme plus tard de transformer l'application en SaaS

A partir de ces éléments nous avons du choisir une méthode de gestion de projet qui permet de travailler progressivement et de pouvoir changer certaines priorite au cours du projet

Methode choisie :

Nous avons choisi de travailler avec la methode Scrum

Scrum est une methode agole qui permet de decouper le projet en plusieurs petites periodes de travail appeler Sprint

Pour ProjetGlobe nous avons choisi des sprints de deux semaines

Chaque sprint doit avoir un objectif precis et permettre de faire avancer une partie du produit

Cette facon de travailler ,nous parait adapter car nous ne cherchons pas a tour definir et tout developper des le debut

Le projet peut evoluer selon les retours du directeur ou des utilisateurs et le backlog peut etre modifier si certaines fonctionnalites deviennent plus importante

Pourquoi Scrum correspond a ProjetGlobe :

Le premier point important est que le projet doit durer plusieurs semaines voir mois il est donc difficile de savoir aujourd'hui exactement comment sera l'intranet dans plusieurs semaines

Scrum permet de travailler par version et de livrer progressivement les fonctionnalite les plus importantes

Le deuxième point concerne les utilisateurs

L'intranet vas etre utiliser par plusieurs types de personnes comme les etudiants les accompagnateurs et les responsable ou encore les intervenants

Les besoins de ces utilisateurs peuvent etre differents et leurs retours peuvent faire evoluer certaines fonctionnalites

Avec Scrum nous pouvons presenter regulierement lavancement du projet pendant les Sprints Reviex et recuperer les retour du directeur

Le troisieme point concerne le futur du produit

Le sujet indique que l'application pourra peut etre devenir un SaaS

Nous avons donc besoin d'une methode qui permet de faire evoluer le produit sans devoir connaitre des maintenant toutes les fonctionnalites futures

Recommandation : Scrum, en sprints de 2 semaines, avec une revue de sprint présentée au directeur.
Le projet dure plusieurs mois, sera livré en plusieurs versions et ses besoins vont évoluer jusqu'au SaaS : une méthode itérative permet de livrer tôt, d'ajuster les priorités à chaque sprint et de faire vivre la documentation avec le produit.

1. Detaille du choix de Scrum pour ce projet

Caractéristique du projet	Ce que Scrum apporte
Projet sur plusieurs années, besoins appelés à changer	Le backlog est repriorité à chaque sprint : un nouveau besoin du directeur entre sans renégocier un contrat figé
Plusieurs versions annoncées	Un incrément utilisable à chaque sprint ; V1, V2, V3 sont des jalons du backlog
Directeur qui veut « des informations régulières »	Revue de sprint toutes les 2 semaines : il voit le produit, pas un rapport
Utilisateurs variés (étudiants, accompagnateurs, responsables)	Retours de terrain dès les premiers sprints, en recette avec de vrais utilisateurs
Évolution vers un SaaS	Le Product Goal guide l'ensemble ; l'architecture est ajustée au fil des sprints au lieu d'être tout prévue d'avance

Scrum est appliqué selon le Scrum Guide 2020 : trois responsabilités (Product Owner, Scrum Master, Developers), cinq événements et trois artefacts, chacun avec son engagement (Product Goal, Sprint Goal, Definition of Done). Le rôle de Tech lead n'existe pas dans le Scrum Guide : c'est un développeur qui porte la cohérence technique.

2. Méthodes écartées

Méthode	Ce qu'elle impose	Pourquoi elle ne convient pas ici	Ce qu'on en garde
Cycle en V / cascade	Besoins figés au départ, spécifications complètes, recette à la fin	Le directeur annonce luimême que le projet va évoluer (versions, SaaS) ; on ne verrait rien de concret avant des mois, et un changement coûterait une reprise de documents figés	La traçabilité exigence → conception → test
Kanban seul	Flux continu, limitation du travail en cours, pas d'itérations	Pas de rythme de revue avec le directeur ni d'objectif de version ; adapté à un flux de demandes, moins à la construction d'un produit neuf	Le tableau visuel ; Kanban sera retenu pour la maintenance après la V3
SAFe	Coordination de plusieurs équipes agiles, trains de release	Trop lourd pour une seule équipe de 6 personnes	À réétudier si le SaaS fait grandir l'organisation
XP (eXtreme Programming)	Pratiques d'ingénierie : TDD, binage, intégration continue	Pas un cadre de gestion de projet complet (rôles, pilotage)	Intégration continue, revue de code, tests automatisés, inscrits dans la DoD

3. Organisation Scrum

Rôles (détail dans `organigramme.pdf`) : un Product Owner, un Scrum Master et 4 développeurs, dont le Tech lead. Le directeur est sponsor ; les responsables pédagogiques sont les utilisateurs clés consultés par le PO.

Les evenement Scrum :

Le Sprint Planning est organise au debut de chaque sprint

Le Product Owner presente les elements les plus importants du backlog

Lequipe choisit ensuite les elements quelle pense pouvoir realiser pendant les deux semaines

Un Sprint Goal est defini pour donner un objectif commun au sprint

Daily Scrum :

Un Daily Scrum est réalisé régulièrement pendant le sprint

Il permet de savoir où en est l'équipe et de faire ressortir rapidement les blocages

Le but est de garder une bonne communication sans organiser une longue reunion

Sprint Review :

La Sprint Review est réalisée à la fin du sprint

L'équipe présente ce qui a ete fait

Le directeur et les autres parties prenantes peuvent donner leurs retours

Le Product Owner peut ensuite prendre en compte ces retours dans la priorisation du backlog

Retrospective

La rétrospective permet à l'équipe de faire un bilan de sa façon de travailler

Nous pouvons identifier ce qui fonctionne bien et ce qui doit etre ameliorer pour le prochain sprint

Gestion du Product Backlog :

Le Product Backlog contient les besoins et fonctionnalités prévues pour ProjetGlobe

Les fonctionnalités sont principalement exprimées sous forme de User Stories

Une User Story permet de décrire simplement le besoin du point de vue d'utilisateur

Le Product Owner est responsable de ordre et de la priorisation du backlog

Pour notre projet nous utilisons aussi la méthode MoSCoW afin de separer les besoins selon leur importance

Événement	Quand	Durée	Participants	Sortie
Sprint	2 semaines		Équipe Scrum	Un incrément conforme à la DoD
Sprint Planning	1er jour du sprint	2 h	Équipe Scrum	Sprint Goal + Sprint Backlog
Daily Scrum	Chaque jour	15 min	Developers (+ SM)	Plan des prochaines 24 h, blocages remontés
Sprint Review	Dernier jour	1 h	Équipe + directeur + utilisateurs clés	Démonstration, backlog ajusté
Sprint Retrospective	Après la review	45 min	Équipe Scrum	1 à 2 actions d'amélioration
Affinage du backlog	En continu, 1 h par semaine	—	PO + Developers	Stories prêtes, estimées

Artefact	Engagement	Responsable
Product Backlog	Product Goal	Product Owner
Sprint Backlog	Sprint Goal	Developers
Incrément	Definition of Done	Developers, garantie par le Tech lead

Outils : GitHub (code, pull requests, tickets), GitHub Projects (tableau Scrum : À faire, En cours, En revue, Terminé), intégration continue GitHub Actions.

4. Note de cadrage

La V1 est livrable en environ 9 semaines (un sprint 0 d'une semaine puis 4 sprints), pour un effort estimé à 247 jourshomme. Les montants reposent sur des hypothèses à valider par le directeur.

Dimension	Cadrage
Objectif	Product Goal : un intranet unique où l'étudiant gère son parcours projet et où l'équipe pédagogique pilote ses promotions selon son rôle, sur une architecture prête pour le SaaS
Démarche	Analyse des besoins → exigences (cahier des charges, Product Goal) → conception (BDD,

Dimension	Cadrage
	classes) → développement itératif avec documentation justintime
Délais	Sprint 0 (1 semaine) : environnement, CI, maquettes. V1 = 4 sprints de 2 semaines (110 points Must, vélocité supposée de 28 points). V2 = 2 sprints (44 points Should et Could). V3 SaaS : à réestimer après la V2
Ressources	PO 1 ETP, SM 0,5 ETP, Tech lead 1 ETP, 3 développeurs 3 ETP, soit 5,5 ETP ; un 4e développeur recruté après un an (Job 3)
Budget V1	Sprint 0 : 5 jours × 5,5 ETP ≈ 27 jh ; 4 sprints : 40 jours × 5,5 ETP = 220 jh ; total ≈ 247 jh. Au coût interne supposé de 400 € par jour : ≈ 99 k€, hébergement non compris
Qualité	Definition of Done sur chaque story, recette à chaque version, conformité RGPD et RGAA 4.1.2, couverture de tests progressive (section 6)
Indicateurs	Vélocité par sprint, part des stories acceptées en review, anomalies trouvées en recette, score d'audit d'accessibilité

Risque	Effet	Réponse
Besoins encore flous (temps de log, intervenants)	Reprise de développement	Décisions de cadrage écrites dans le cahier des charges, revues en Sprint Review
Utilisateurs peu disponibles pour la recette	Retours tardifs	Créneau de recette fixé à chaque Sprint Review
Dépendance à Google pour la connexion	Indisponibilité totale si Google tombe	Risque accepté en V1, réévalué pour le SaaS
Dérive du périmètre	Retard de la V1	Tout nouveau besoin passe par le PO et le backlog, jamais directement en sprint

5. Conséquence sur la documentation

Une méthode prédictive fige une documentation complète en amont ; avec Scrum, on fige seulement ce qui engage et ce qui sert de référence commune, et on laisse vivre ce qui décrit le code. **La frontière : ce qui exprime un besoin ou une décision est figé et versionné ; ce qui décrit une implémentation naît avec le code et change avec lui.**

Figé en amont (révisé seulement en Sprint Review, avec numéro de version)	Vivant (mis à jour dans le ticket qui change le code)
Cahier des charges : exigences, matrice des droits, RGPD, RGAA	Documentation de l'API (OpenAPI / Swagger)
Product Goal et découpage en versions	Catalogue des composants (Storybook)
Choix de méthode et organigramme	Schéma de base de données (migrations, MCD et DBML régénérés)
MCD et diagramme de classes de référence de la V1	Diagramme de classes détaillé, décisions d'architecture (ADR)
Stratégie de tests	Cahier de recettes : un scénario par story livrée
	README, CHANGELOG, couverture de tests

Pourquoi figer la conception de la V1 : c'est la conception commune qui permet à l'équipe de démarrer sans se contredire. Elle devient ensuite vivante : chaque changement de schéma passe par une migration et met à jour les scripts du modèle dans le même ticket.

6. Plan de gestion de la documentation justintime

Il n'y a pas encore de code, donc pas encore de documentation justintime : ce plan dit comment elle sera produite et tenue à jour. Chaque document répond aux quatre questions ; la garantie est toujours un critère de la Definition of Done ou un contrôle automatique.

Document	Quand estil produit ou mis à jour ?	Qui en est responsable ?	Où et avec quel outil ?	Qu'estce qui garantit qu'il reste à jour ?
Documentation API (OpenAPI / Swagger)	À chaque route créée ou modifiée, dans le même ticket	Le développeur du ticket, vérifié en revue par le Tech lead	Générée depuis les annotations du code, publiée sur `/api/docs`	DoD ; la CI échoue si une route n'est pas documentée
Catalogue de composants (Storybook)	À chaque composant d'interface créé ou modifié	Le développeur front du ticket	Storybook, publié à chaque fusion sur la branche principale	DoD ; vérification d'accessibilité automatique sur chaque story

Document	Quand estil produit ou mis à jour ?	Qui en est responsable ?	Où et avec quel outil ?	Qu'estce qui garantit qu'il reste à jour ?
Schéma de base de données	À chaque migration	Le développeur du ticket, validé par le Tech lead	Migrations dans le dépôt ; `mcd.mcd` et `mpd_dbdiagram.dbml` régénérés dans `/modeles`	DoD ; pas de fusion d'une migration sans mise à jour des scripts
Décisions d'architecture (ADR)	À chaque choix structurant (bibliothèque, sécurité, découpage)	Le Tech lead	`/docs/adr`, un fichier Markdown par décision	Revue à la rétrospective ; une décision non écrite n'est pas appliquée
Couverture de tests	À chaque pull request	Les développeurs, seuil porté par le Tech lead	Rapport de couverture dans la CI	Seuil progressif bloquant : 60 % en V1, 70 % en V2, 80 % en V3
Cahier de recettes	À chaque story livrée : un scénario RECxx par critère d'acceptation	Le PO, avec le Tech lead	`plan_tests_recette` dans le dépôt	DoD ; une story sans scénario passé n'est pas « terminée »
README	À chaque changement d'installation ou de configuration	Le développeur du ticket	Racine du dépôt	DoD ; test d'installation par chaque nouvel arrivant (onboarding)
CHANGELOG	À chaque version	Le Scrum Master	`CHANGELOG.md`	Critère de sortie de version, présenté en Sprint Review

Place dans la planification : au Sprint Planning, le temps de documentation est compté dans l'estimation de chaque story ; en Sprint Review, le PO vérifie que le cahier de recettes et le CHANGELOG suivent l'incrément ; en rétrospective, l'équipe vérifie que la documentation a suivi le code.

7. Definition of Done

Nous avons defini une Definition of Done commune afin de savoir a quel moment une User Story peut vraiment etre consideree comme terminer

Une story est terminée quand tous ces critères sont remplis ; sinon elle retourne au backlog et n'est pas présentée en Sprint Review.

- les criteres acceptation sont respecter
- Le developpement necessaire est terminer
- Le code a ete relu
- Les tests prevue pour la fonctionnalite sont passer
- Aucun probleme bloquant connnu reste sur les fonctionnalités
- La documentation liee a cette fonctionnalite est mise a jour
- La fonctionnalite est disponible sur environnement de recette
- Le resultat fonctionnel peut etre valide par le Product Owner

Cette definition of done permt aussi de ne pas produire toute la documentation technique au debut du projet

La documentation proche du code est produite au moment ou les fonctionnalites sont developpee
Cela evite de creer des documents plusieurs mois avant le developpement puis de devoir tout refaire quand les choix techniques evoluent

Critères de sortie d'une version (V1, V2, V3) : toutes les exigences de la version validées en recette, aucune anomalie bloquante ouverte, CHANGELOG à jour, déclaration d'accessibilité mise à jour.